

Classificazione Prezzi Smartphone

Gruppo di lavoro

Andrea Fortunato, MAT. 670775, a.fortunato42@studenti.uniba.it

[Link al progetto](#)

A.A. 2025-2026

Introduzione

L'obiettivo del progetto è quello di applicare alcune delle conoscenze maturate durante le lezioni di Ingegneria della Conoscenza, applicando algoritmi visti in forma teorica.

In particolare per questo progetto ho usato un dataset presente sul sito Kaggle chiamato “[Mobile Price Classification](#)”, diviso a priori in un dataset di training e uno di test nella quale è completamente assente la feature da predire.

Avendo a disposizione un training set con i dati dei telefoni cellulare e il loro price range (quattro valori da 0 a 3), l'obiettivo è prevedere il price range dei telefoni presenti nel dataset di test.

Argomenti trattati

[Descrizione del dataset](#)

[Ragionamento logico](#)

[Apprendimento Non Supervisionato](#)

[Apprendimento Supervisionato](#)

[Valutazione delle prestazioni](#)

[Curve di apprendimento](#)

[Conclusioni](#)

Descrizione del dataset

Il dataset utilizzato è formato dalle seguenti features:

- battery_power: capacità della batteria misurata mAh
- blue: variabile booleana che indica se il telefono ha la funzionalità Bluetooth
- clock_speed: velocità del microprocessore nell'eseguire istruzioni
- dual_sim: variabile booleana che indica se il telefono può utilizzare due sim contemporaneamente
- fc: mega pixels della fotocamera frontale
- four_g: variabile booleana che indica se il telefono ha la funzionalità 4G
- int_memory: memoria interna del telefono in Gigabytes
- m_dep: spessore del telefono in centimetri
- mobile_wt: peso del telefono
- n_cores: numero delle unità di elaborazione
- pc: mega pixels della fotocamera principale
- px_height: altezza della risoluzione in pixel
- ram: Megabytes di memoria RAM
- sc_h: altezza dello schermo in centimetri
- sc_w: larghezza dello schermo in centimetri
- talk_time: durata di una intera carica di batteria in chiamata
- three_g: variabile booleana che indica se il telefono ha la funzionalità 3G
- touch_screen: variabile booleana che indica se il telefono ha il touch screen o meno
- wifi: variabile booleana che indica se il telefono ha il wifi o meno
- price_range: feature obiettivo che indica il range del costo del telefono cellulare (0=basso costo, 1=medio costo, 2=alto costo, 3=altissimo costo)

Ragionamento logico

In questo progetto il programma crea automaticamente tramite la libreria `pyswip` la Knowledge Base partendo dal dataset di train. In particolare, per ogni telefono crea un fatto contenente tutte le informazioni su quel telefono, così schematizzato:

```
% telefono(battery_power, blue, clock_speed, dual_sim, fc, four_g, int_memory,
m_dep, mobile_wt, n_cores, pc, px_height, px_width, ram, sc_h, sc_w, talk_time,
three_g, touch_screen, wifi, price_range)
```

Altri fatti sono i range dei possibili valori per ogni caratteristica in ciascuna fascia di prezzo (escluse le variabili booleane che avranno valori pari a 0 o 1). Per esempio, la caratteristica “ram” ha i seguenti fatti:

```
range_ram(0, 256, 1974).
```

```
range_ram(1, 387, 2811).
```

```
range_ram(2, 1185, 3916).
```

```
range_ram(3, 2259, 3998).
```

Quindi questo significa che per un telefono di fascia 0, i valori di ram andranno da 256 a 1974 e così via.

Questi fatti insieme alle seguenti regole vengono utilizzati per creare degli esempi sintetici di telefoni.

```
:- use_module(library(random)).
```

```
genera_intero(Min, Max, X) :- random(Min, Max, X).
```

```
genera_decimale(Min, Max, Out) :- random(Min, Max, X), Out is round(X * 10) / 10.0.
```

Le regole precedenti servono per generare numeri casuali interi e decimali nel range Min e Max. Per i numeri decimali nel caso del nostro dataset è interessante solo la prima cifra decimale e per questo arrotondo.

Un'altra regola utile alla creazione di telefoni con caratteristiche realistiche è la seguente:

```
vincolo_connettivita(FourG, ThreeG) :- (FourG == 1 -> ThreeG = 1 ; random(0, 2, ThreeG)).
```

Questa regola permette alla Knowledge Base di stabilire che se un telefono ha la tecnologia 4G, in automatico avrà anche la tecnologia 3G. Ossia se FourG equivale ad 1, allora anche ThreeG dovrà essere necessariamente 1, altrimenti il valore di ThreeG sarà 0 o 1.

Infine, utilizzando la regola “*genera_nuovo_telefono*(PR, telefono(...)) :- ...” presente in forma completa sul file `knowledge_base.pl`, passando al posto di PR il `price_range` desiderato, la query restituirà un nuovo telefono di quella fascia di prezzo.

Ecco alcuni esempi:

```
(Fascia 0): {'Telefono': 'telefono(1438, 0, 1.2, 0, 18, 0, 59, 0.9, 126, 5, 14, 451, 995, 1519, 12, 9, 11, 1, 1, 1, 0)'}  
(Fascia 1): {'Telefono': 'telefono(560, 0, 1.8, 1, 14, 0, 43, 0.5, 160, 1, 18, 1422, 756, 1767, 17, 16, 13, 0, 1, 0, 1)'}  
(Fascia 2): {'Telefono': 'telefono(1485, 1, 1.9, 0, 3, 1, 3, 0.4, 132, 5, 0, 237, 1623, 1192, 10, 11, 7, 1, 1, 1, 2)'}  
(Fascia 3): {'Telefono': 'telefono(1154, 0, 3.0, 1, 14, 1, 52, 0.6, 134, 1, 8, 847, 630, 2577, 9, 14, 3, 1, 1, 1, 3)'}
```

Quindi, utilizzando una query sulla regola “genera_nuovo_telefono” è possibile creare un dataset sintetico per effettuare per esempio over-sampling o per utilizzare tecniche di validazione che funzionano meglio su un numero di dati maggiore.

Apprendimento non supervisionato

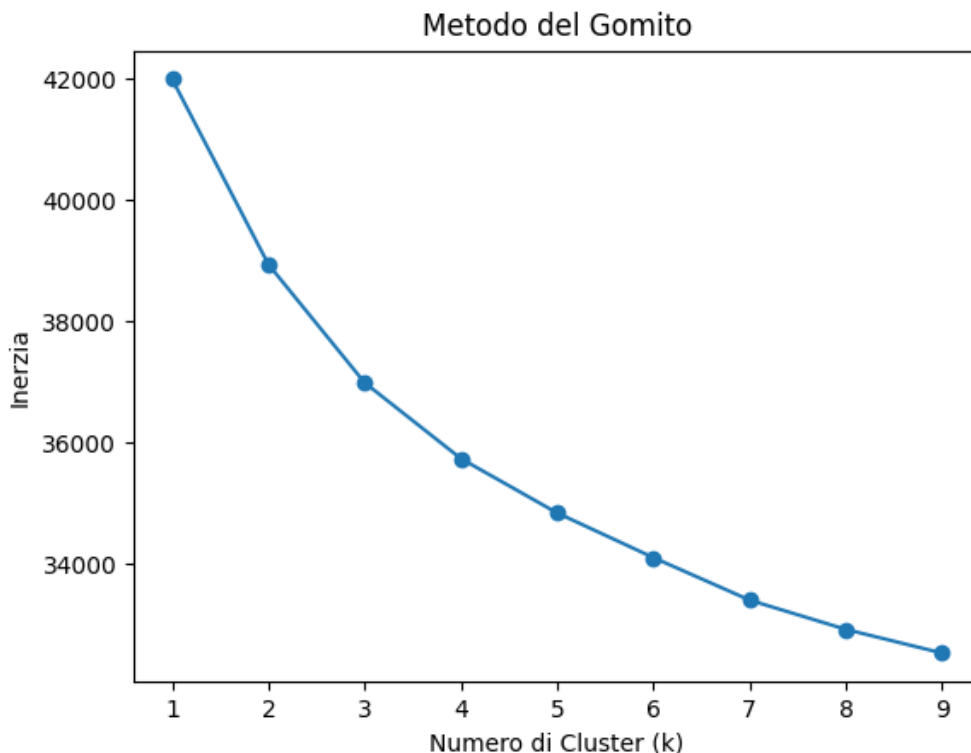
Avendo a disposizione un dataset supervisionato, ossia con i valori di `price_range` già assegnate ai dati, ho deciso di utilizzare l'algoritmo di apprendimento non supervisionato KMeans per verificare la presenza di sottoclassi all'interno di ogni classe di prezzo.

Per utilizzarlo ho eliminato e diviso il dataset in quattro parti, ognuna delle quali rappresenta una classe di prezzo (0, 1, 2, 3) ed ho rimosso la colonna `price_range`.

Per utilizzare questo algoritmo è necessario trovare il numero ideale di cluster in cui è suddiviso il dataset.

Per determinare il numero reale di cluster, ho deciso di usare il "Metodo del Gomito": il programma disegna su un grafico il variare dell'inerzia all'aumentare del numero di cluster su cui si effettua il KMeans. L'inerzia è la somma dei quadrati delle distanze tra ogni punto del dataset e il centroide del cluster a cui appartiene e nel grafico è indicata sull'asse delle Y. Il numero di cluster è indicato sull'asse delle X.

Per tutte le quattro sezioni del dataset ho ottenuto il valore 4. Ecco un esempio di grafico ottenuto:



Quindi il gomito del grafico indica che 4 è il numero ideale di cluster per il mio dataset. In particolare, in Python ho utilizzato la classe `KneeLocator` della libreria `kneed` inizializzata con il range su cui testare, i valori di inerzia, il parametro `"curve"` che per il valore `'convex'` rileva il gomito e il parametro `"direction"` che indica la direzione della curva. Inizializzando la classe con questi valori, l'attributo `"elbow"` assume il valore 4 (come è visibile nel grafico precedente).

```
inertia = []
maxK = 10
for i in range(1, maxK):
    km = KMeans(n_clusters=i, n_init=5, init='random')
    km.fit(x_scaled)
    inertia.append(km.inertia_)

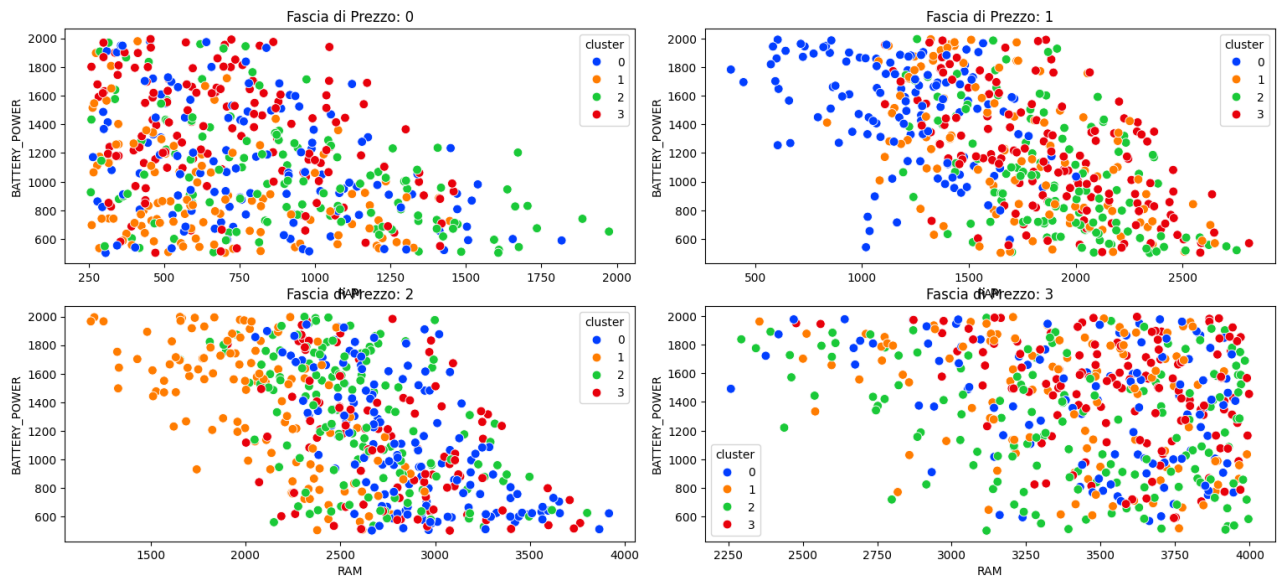
from kneed import KneeLocator
kl = KneeLocator(range(1, maxK), inertia, curve='convex', direction='decreasing')
k_ideale = kl.elbow
```

Dato che all'interno del dataset ci sono dati con unità di misura diverse, ho deciso di scalare i dati utilizzando la classe `StandardScaler` della libreria `sklearn`, per minimizzare la distanza euclidea.

La scelta di utilizzare il metodo di clustering `KMeans` per vedere la distribuzione dei dati all'interno delle fasce di prezzo, si è rivelata abbastanza fallimentare poiché come evidenza il grafico che segue, i cluster sono molto omogenei poiché i punti sono molto vicini tra loro. Questo significa che per ogni fascia di prezzo avremo telefonini con caratteristiche molto simili tra loro.

Sebbene il clustering sia stato effettuato tenendo conto di tutte le variabili all'interno del dataset, per visualizzare in modo bidimensionale ho scelto di proiettare sugli assi `'ram'` e `'battery_power'` poiché sono le variabili più rappresentative.

Analisi Cluster per Fascia di Prezzo (Basata su RAM e Batteria)



Apprendimento Supervisionato

L'apprendimento supervisionato è una branca dell'apprendimento automatico, il cui obiettivo principale è far sì che l'algoritmo impari una relazione tra i dati di input e output in modo tale da poter fare previsioni accurate su dati mai visti prima.

Nel mio caso l'apprendimento supervisionato si occupa di un problema di classificazione, ossia predire un valore per l'attributo `price_range` compreso nell'insieme finito prefissato $\{0, 1, 2, 3\}$.

Per fare ciò ho utilizzato e confrontato tra loro tre modelli di apprendimento supervisionato: Decision Tree, Random Forest e Logistic Regression

Decision Tree

Il Decision Tree è un albero binario costituito da nodi interni etichettati con le condizioni sulle feature di input e nodi foglia etichettati con una stima puntuale per la feature obiettivo. Quindi per un dato esempio di feature di input, partendo dalla radice e valutando ogni condizione incontrata nei nodi interni si arriverà ad un nodo foglia contenente la stima puntuale.

In particolare, ho usato la classe `DecisionTreeClassifier` della libreria `sklearn` con parametro `max_depth = 5`, ossia altezza (o profondità) dell'albero massima uguale a 5. Ho fatto questa scelta perché altrimenti l'albero sarebbe difficile da leggere e visualizzare, ben consapevole di perdere parecchia accuratezza nelle predizioni. Il programma mostra l'albero generato e ne fornisce anche una versione testuale.

Per la validazione del modello ho usato la `k-fold cross validation` sul training set usando il Decision Tree generato ed ho ottenuto un'accuratezza media stimata pari all'82.35%.

Random Forest

La Random Forest è un modello che utilizza un numero definito di alberi di decisione addestrati su parti differenti del dataset di training.

In particolare, ho usato la classe `RandomForestClassifier` della libreria `sklearn` con parametro `n_estimators = 100`, ossia la foresta sarà formata da 100 alberi di decisione e `max_depth = 10`, ossia altezza degli alberi pari a 10.

La Random Forest ha ottenuto un'accuratezza media stimata pari all'89.25%.

Logistic Regression

La Logistic Regression è un modello che cerca di prevedere la probabilità che un dato appartenga a una determinata classe. Durante l'addestramento impara quanto "pesano" le feature.

In particolare, ho usato la classe LogisticRegression della libreria sklearn con parametro `max_iter = 1000`, ossia l'algoritmo userà 1000 iterazioni durante il processo di ottimizzazione.

Per `price_range = 3`, ossia la classe più alta dei prezzi, i pesi di alcune features sono i seguenti:

	Feature	Coefficient
13	ram	10.858849
0	battery_power	2.757012
12	px_width	1.584425
11	px_height	1.579760
6	int_memory	0.248160
4	fc	-0.014161
2	clock_speed	-0.045522
7	m_dep	-0.099580
19	wifi	-0.102515
8	mobile_wt	-0.540191

Quindi il modello considera i valori di ram, battery_power, px_width, px_height molto importanti per telefoni costosi; invece, le feature sottostanti saranno presenti sia in telefoni più economici che in quelli costosi (dato che sono vicini allo 0).

Questo modello ha ottenuto un'accuratezza media stimata pari al 97.50%.

È più accurato di modelli come il Decision Tree perché questi al contrario della Logistic Regression danno importanza solo ad alcune variabili (in questo caso ram).

Valutazione delle prestazioni

Dato che il dataset di Test è privo della feature obiettivo “price_range” per valutare i modelli ho utilizzato la K-Fold Cross Validation che addestra il modello diverse volte (nel mio caso 5) utilizzando ogni volta sezioni diverse di esempi di training e test all’interno del Training set.

In particolare, per valutare le prestazioni ho deciso di utilizzare le seguenti metriche: accuracy, precision_macro, recall_macro e f1_macro.

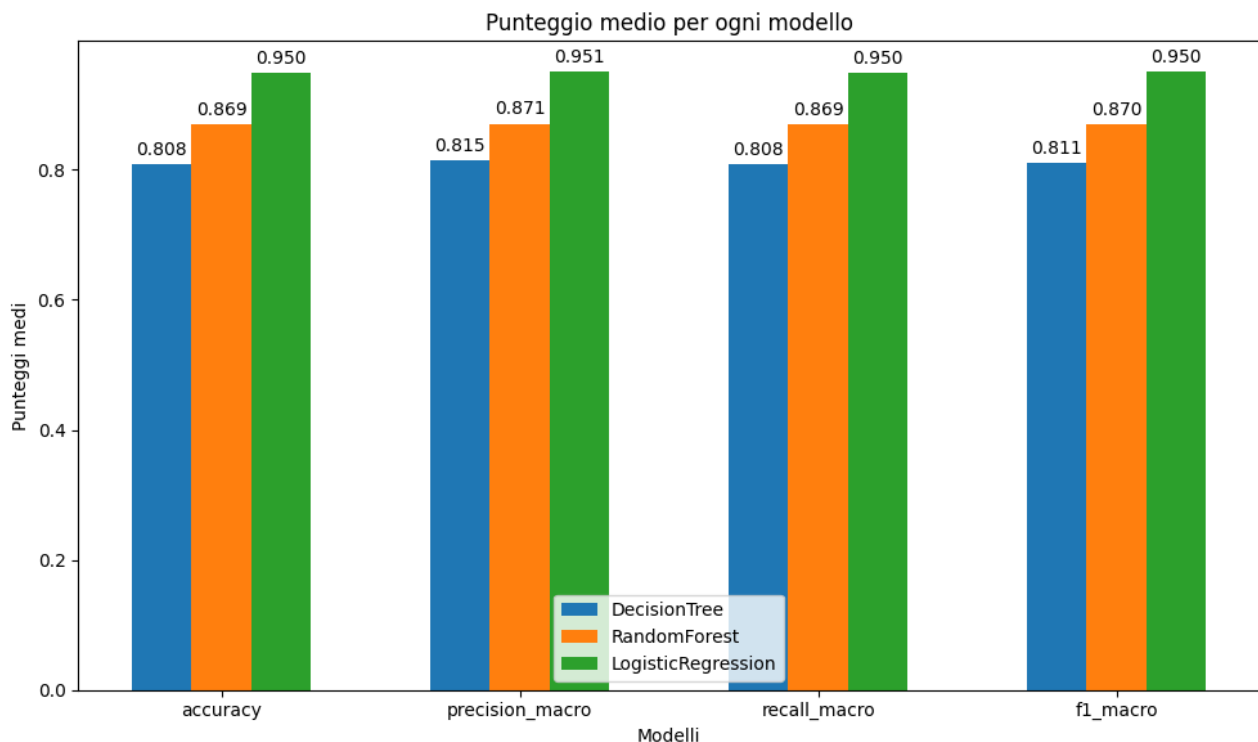
L’accuracy rappresenta la frazione di previsioni corrette rispetto al totale delle previsioni.

La precision_macro è una metrica che calcola la precisione per ogni classe (price_range = 0, 1, 2, 3), ossia il numero di esempi classificati in una classe C sul numero di istanze della classe C (precision), e ne calcola la media.

Il recall_macro è una metrica che per ogni classe calcola il numero di istanze di classe C classificate nella classe C sul numero di istanze della classe C (recall), e ne calcola la media.

La f1_macro calcola per ogni classe la media armonica tra precision e recall (f1) e ne calcola la media.

Nel valutare il mio sistema ho ottenuto i seguenti risultati:



Questo ci fa capire che dei tre modelli la Logistic Regression sembrerebbe essere il modello più affidabile nel fare previsioni su dati mai visti prima. Ma valori delle metriche vicino all'1 potrebbero anche essere sintomo di Sovradattamento (overfitting).

Per Sovradattamento si intende la tendenza dei modelli di apprendimento a imparare correlazioni spurie nei dati che non si riflettono sull'intero dominio del problema. Questo comporta la possibilità che il modello non sarà in grado di predire le classi di appartenenza di esempi mai visti, in assenza di queste correlazioni (comportando errori e diminuendo l'affidabilità del modello).

Per scongiurare la presenza di Sovradattamento ho preso in considerazione la varianza, la deviazione standard e le curve di apprendimento.

La Varianza è la media dei quadrati tra ciascun dato e la media.

La Deviazione Standard è la radice quadrata della Varianza.

La Varianza indica quanto cambiano le predizioni del modello se questo viene addestrato su un gruppo di dati diverso. Se alta potrebbe essere sintomo di Sovradattamento, ossia il modello potrebbe aver imparato "a memoria" i dati di training e una volta testato con dati nuovi sbaglierebbe nel fare predizioni.

Quindi per validare i tre modelli ho usato la classe RepeatedKfold della libreria sklearn effettuando 5 ripetizioni su 5 fold diversi ogni volta.

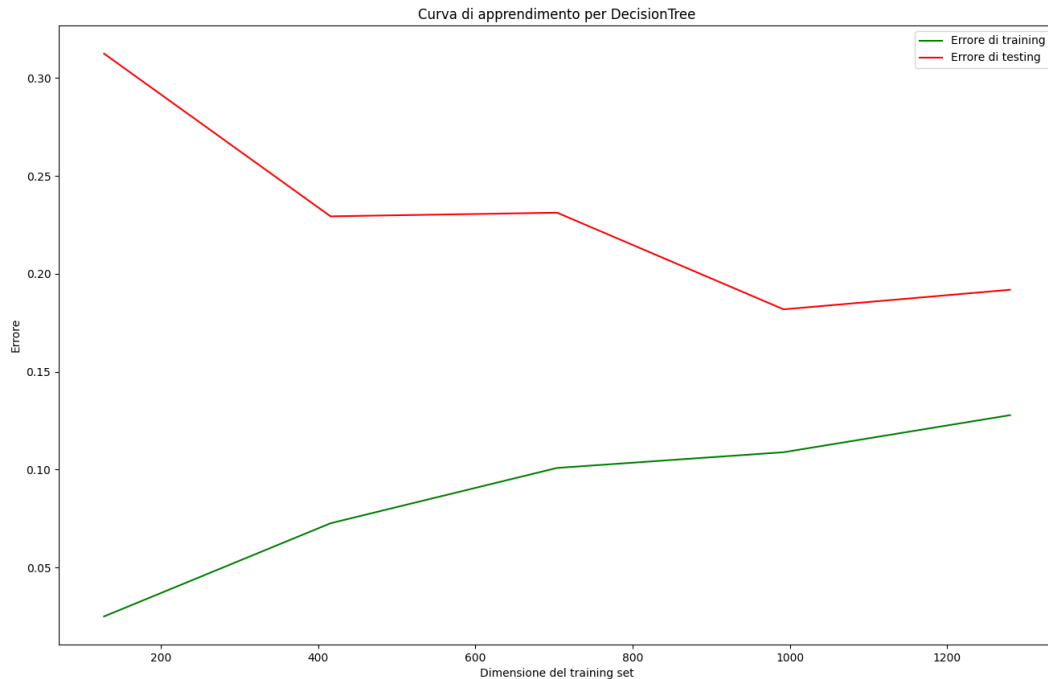
Numero Ripetizione	Numero Fold	DecisionTree	RandomForest	LogisticRegression
1	1	0.843750	0.859375	0.934375
	2	0.846875	0.865625	0.953125
	3	0.796875	0.875000	0.971875
	4	0.821875	0.868750	0.940625
	5	0.828125	0.887500	0.943750
2	1	0.837500	0.875000	0.971875
	2	0.840625	0.856250	0.928125
	3	0.825000	0.831250	0.956250
	4	0.853125	0.859375	0.940625
	5	0.790625	0.875000	0.956250
3	1	0.840625	0.887500	0.956250
	2	0.834375	0.862500	0.975000
	3	0.850000	0.831250	0.937500
	4	0.840625	0.884375	0.937500
	5	0.831250	0.871875	0.943750
4	1	0.821875	0.903125	0.946875
	2	0.865625	0.878125	0.946875
	3	0.868750	0.878125	0.965625
	4	0.793750	0.862500	0.940625
	5	0.837500	0.856250	0.962500
5	1	0.825000	0.890625	0.953125
	2	0.878125	0.875000	0.965625
	3	0.840625	0.900000	0.956250
	4	0.796875	0.887500	0.981250
	5	0.853125	0.884375	0.925000

	Media	Deviazione Standard	Min	Max
DecisionTree	0.834500	0.022687	0.790625	0.878125
RandomForest	0.872250	0.017800	0.831250	0.903125
LogisticRegression	0.951625	0.014909	0.925000	0.981250

I valori di Deviazione Standard indicano che la Logistic Regression è il modello piu' affidabile, poiché i risultati sono molto consistenti tra un fold e l'altro. Inoltre, si può notare come i valori variano di poco tra un fold e l'altro con tutti e tre i modelli, il che significa che i modelli non sono molto sensibili ai dati di addestramento e non è presente sovraddattamento.

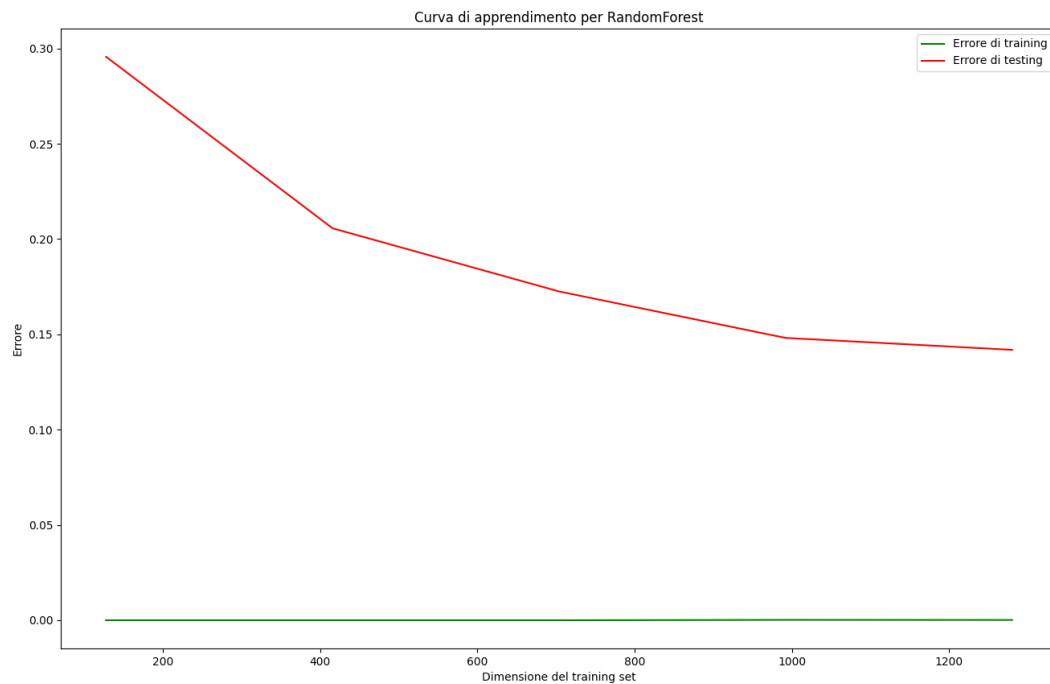
Curve di apprendimento

Decision Tree



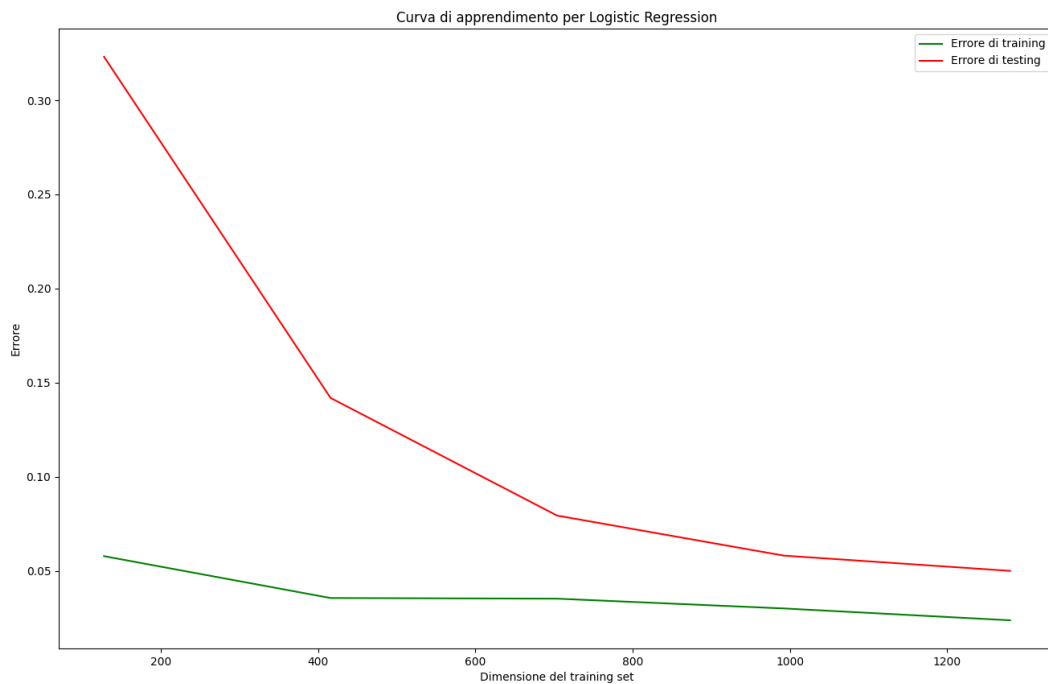
Il grafico mostra che le due curve tendono ad avvicinarsi per valori abbastanza lontani dallo 0. Questo significa che il modello potrebbe essere troppo semplice, ossia anche aumentando il numero di dati su cui addestrare, questo modello difficilmente riuscirà a distinguere le diverse classi di valori per `price_range`. Una possibile soluzione potrebbe essere quella di aumentare l'altezza dell'albero (`max_depth`).

Random Forest



Il grafico mostra che le due curve tendono a rimanere parecchio distanti tra loro, inoltre la curva di training è molto vicina allo 0. Questo è sintomo del fatto che il modello durante la fase di apprendimento tende ad imparare “a memoria” il training set e non riesce a generalizzare (Sovradattamento).

Logistic Regression



Il grafico mostra che le due curve tendono ad avvicinarsi e i valori di errore ad abbassarsi. Questo è sintomo del fatto che il modello ha imparato le regole generiche di classificazione e quindi potremo aspettarci delle ottime predizioni sul dataset di test.

Conclusioni

Con il presente progetto mi ero posto l'obiettivo di confrontare diversi algoritmi di classificazione per scegliere quello piu' adatto nel predire la classe di prezzo per i telefoni presenti nel file test.csv.

Partendo da un'analisi esplorativa tramite K-Means è stato possibile confermare che in assenza della feature obiettivo price_range, i dati all'interno delle quattro classi di prezzo sono omogenei.

Per l'obiettivo come era noto già a priori, è piu' adatto usare modelli di apprendimento supervisionato, con task di classificazione.

Tra i modelli presi in esame la Logistic Regression è risultato il modello superiore per questo specifico compito. La capacità del modello di dare peso a feature piu' importanti come i valori di "ram" o "battery_power" ha permesso di ottenere un modello equilibrato, con varianza contenuta e un'ottima capacità di generalizzazione.

Sviluppi Futuri

Sebbene i risultati siano soddisfacenti, il progetto potrebbe essere ulteriormente esteso attraverso:

1. **Scelta degli iper-parametri:** utilizzando la strategia di *GridSearch* per ottimizzare la scelta degli iper-parametri in modo tale da ottenere migliori risultati da modelli come Decision Tree e Random Forest.
2. **Ragionamento probabilistico:** utilizzando algoritmi di apprendimento supervisionato basati sul Teorema di Bayes, si potrebbe ottenere la probabilità di appartenenza ad una classe di prezzo di dati forniti in input